📚 Programming in Python — Comprehensive Study Notes

> ***Syllabus Coverage:*** *100% (Unit 1 & Unit 2)* ***Subject:*** *Programming in Python* ***Code:*** *CIE-332T / IOT-320T* ***Exam Priority:*** *🔥 High Priority Topics Marked | ⭐ Medium Priority | 📚 Standard*

📋 Table of Contents

- Unit 1: Introduction to Python & Flow Control
  - Topic 1.1: Introduction to Python Programming 🔥
  - Topic 1.2: Python Basics - Data Types 🔥
  - Topic 1.3: String Concatenation and Replication ⭐
  - Topic 1.4: Variables and Assignment 🔥
  - Topic 1.5: Dissecting Your Program 📚
  - Topic 1.6: Flow Control - Boolean Values & Operators 🔥
  - Topic 1.7: Comparison and Boolean Operators 🔥
  - Topic 1.8: Elements of Flow Control 🔥
  - Topic 1.9: Program Execution 📚
  - Topic 1.10: Flow Control Statements 🔥
  - Topic 1.11: Importing Modules ⭐
  - Topic 1.12: Ending a Program Early with sys.exit() 📚
- Unit 2: Functions, Lists, Dictionaries & Strings
  - Topic 2.1: Functions - Definition and Parameters 🔥
  - Topic 2.2: Return Values and Return Statements 🔥
  - Topic 2.3: The None Value ⭐
  - Topic 2.4: Keyword Arguments and print() 🔥
  - Topic 2.5: Local and Global Scope 🔥
  - Topic 2.6: The global Statement ⭐
  - Topic 2.7: Exception Handling 🔥
  - Topic 2.8: Lists - The List Data Type 🔥
  - Topic 2.9: Working with Lists 🔥
  - Topic 2.10: Augmented Assignment Operators ⭐

- Topic 2.11: List Methods 🔥
- Topic 2.12: Dictionaries - The Dictionary Data Type 🔥
- Topic 2.13: Pretty Printing 📚
- Topic 2.14: Using Data Structures to Model Real-World Things ⭐
- Topic 2.15: Manipulating Strings 🔥
- Topic 2.16: Useful String Methods 🔥

- Other Important Concepts (Frequently Asked in Exams) 🔥
  - Topic 3.1: Tuples 🔥
  - Topic 3.2: Sets ⭐
  - Topic 3.3: List Comprehensions 🔥
  - Topic 3.4: Dictionary Comprehensions 🔥
  - Topic 3.5: Lambda Functions 🔥
  - Topic 3.6: Map, Filter, and Reduce 🔥
  - Topic 3.7: File Handling 🔥
  - Topic 3.8: Regular Expressions 🔥
  - Topic 3.9: Object-Oriented Programming - Classes and Objects 🔥
  - Topic 3.10: Inheritance 🔥
  - Topic 3.11: Generators ⭐
  - Topic 3.12: Decorators ⭐
  - Topic 3.13: Modules and Packages ⭐

Unit 1: Introduction to Python & Flow Control

Topic 1.1: Introduction to Python Programming 🔥

Overview

Python is a high-level, interpreted programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

Key Features of Python

Feature	Description
Interpreted	Executes code line by line, no compilation needed
High-Level	Abstracts away hardware complexity
Object-Oriented	Supports classes, objects, inheritance
Dynamic Typing	No need to declare variable types
Portable	Works on Windows, Mac, Linux
Rich Libraries	Extensive standard library and third-party packages

Python Philosophy

> ***"Readability counts."*** *— Zen of Python*

Python emphasizes code readability and clean syntax. The philosophy can be accessed by running `import this` in Python.

Installing Python

- **CPython:** Official implementation from python.org
- **Anaconda:** Distribution with scientific packages
- **Installation:** Download from python.org for Windows/Mac/Linux

Python Interpreter

The Python interpreter executes Python code. Two modes:

1. **Interactive Mode:** Enter expressions directly in the shell
2. **Script Mode:** Write code in a .py file and execute

First Python Program

```python
print("Hello, World!")
```

Output: `Hello, World!`

Python Versions

- **Python 2.x:** Legacy version (no longer supported)
- **Python 3.x:** Current version (recommended)

Python Development Environments

Type	Examples
IDEs	PyCharm, VS Code, Jupyter Notebook
Text Editors	Sublime Text, Atom, Vim
Online Platforms	Google Colab, Jupyter Online

Key Takeaway

- ✅ Python created by Guido van Rossum in 1991
- ✅ Interpreted, high-level, dynamically typed language
- ✅ "Readability counts" is a core philosophy
- ✅ Python 3.x is the current standard version

Topic 1.2: Python Basics - Data Types 🔥

Overview

Python has several built-in data types for storing different kinds of data.

Data Types Overview

Type	Description	Example
int	Integer (whole numbers)	`5`, `10`, `-3`
float	Floating-point (decimal)	`3.14`, `0.5`, `-2.7`
str	String (text)	`"hello"`, `'Python'`
bool	Boolean (True/False)	`True`, `False`
list	Ordered, mutable sequence	`[1, 2, 3]`
tuple	Ordered, immutable sequence	`(1, 2, 3)`
dict	Key-value pairs	`{"name": "John"}`
set	Unordered unique items	`{1, 2, 3}`

The Integer Data Type (int)

Integers represent whole numbers without any fractional part.

```
x = 5
print(x)            # Output: 5
print(type(x))     # Output: <class 'int'>

# Operations
print(10 + 5)      # Addition: 15
print(10 - 5)      # Subtraction: 5
print(10 * 5)      # Multiplication: 50
print(10 / 5)      # Division: 2.0
print(10 % 3)      # Modulus: 1
print(10 ** 2)     # Exponent: 100
```

Key Features:

- Arbitrary size (no overflow)
- Supports all arithmetic operations

The Floating-Point Data Type (float)

Floating-point numbers represent real numbers with a fractional part.

```
y = 3.14
print(y)           # Output: 3.14
print(type(y))    # Output: <class 'float'>

# Scientific notation
z = 6.022e23
print(z)          # Output: 6.022e+23
```

The String Data Type (str)

Strings represent sequences of characters.

```
s = "hello"
print(s)           # Output: hello
print(type(s))    # Output: <class 'str'>
```

The Boolean Data Type (bool)

Boolean represents truth values.

```
is_valid = True
is_admin = False

print(type(is_valid))  # <class 'bool'>
```

Key Takeaway

- ✅ Python is dynamically typed
- ✅ Integers can be arbitrarily large
- ✅ Strings are immutable (cannot be modified)
- ✅ Use `type()` to check data type

Topic 1.3: String Concatenation and Replication ⭐

Overview

Strings can be combined and repeated using specific operators.

String Concatenation

Definition: Concatenation is the process of combining two or more strings into a single string using the `+` operator.

```
str1 = "Hello"
str2 = "World"
result = str1 + " " + str2
print(result)  # Output: Hello World
```

String Replication

Definition: Replication repeats a string a specified number of times using the `*` operator.

```
str1 = "Python"
result = str1 * 3
print(result)  # Output: PythonPythonPython
```

Escape Characters

Escape	Description
`\n`	Newline
`\t`	Tab
`\\`	Backslash
`\'`	Single quote
`\"`	Double quote

Key Takeaway

- ✅ Use `+` for concatenation
- ✅ Use `*` for replication
- ✅ Strings are immutable in Python

Topic 1.4: Variables and Assignment 🔥

Overview

Variables are named containers used to store data in memory.

Variables

```
x = 10              # Integer
name = "John"       # String
pi = 3.14           # Float
is_active = True    # Boolean
```

Variable Naming Rules

Rule	Valid	Invalid
Start with letter or underscore	`name`, `_value`	`1name`
Can contain letters, numbers, underscore	`my_var2`	`my-var`
Case-sensitive	`Name`, `name`	—
Cannot be a keyword	`print_`	`for`, `class`

Multiple Assignment

```
# Assign same value to multiple variables
x = y = z = 0

# Assign different values
a, b, c = 1, 2, 3
```

Constants

Constants are variables meant to not change. By convention, use UPPERCASE names:

```
PI = 3.14159
MAX_SIZE = 100
```

Dynamic Typing

Python determines variable type at runtime:

```
x = 5        # int
x = "hello"  # Now str - no error!
```

Key Takeaway

- ✅ Use `=` for assignment
- ✅ Variables store references to objects
- ✅ Python is dynamically typed
- ✅ Constants use UPPERCASE by convention

Topic 1.5: Dissecting Your Program 📚

Overview

Understanding your program structure through comments, debugging, and documentation.

Comments

Comments explain code and are ignored by the interpreter.

```
# This is a single-line comment

"""
This is a
multi-line comment
using triple quotes
"""
```

Indentation

Python uses indentation to define code blocks:

```
if x > 5:
    print("x is greater than 5")  # Indented block
```

Docstrings

Definition: A docstring is a string literal that documents a module, function, class, or method.

```
def add(a, b):
    """Add two numbers and return the result."""
    return a + b

print(add.__doc__)  # Output: Add two numbers and return the result.
```

Key Takeaway

- ✅ Use `#` for single-line comments
- ✅ Indentation defines code blocks (typically 4 spaces)
- ✅ Docstrings document code using triple quotes

Topic 1.6: Flow Control - Boolean Values & Operators 🔥

Overview

Boolean values and operators control the flow of program execution.

Boolean Values

Boolean values represent truth: `True` or `False`

```
is_active = True
is_admin = False
```

Comparison Operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal	`5 != 3`	`True`
`>`	Greater than	`5 > 3`	`True`
`<`	Less than	`5 < 3`	`False`
`>=`	Greater or equal	`5 >= 5`	`True`
`<=`	Less or equal	`5 <= 3`	`False`

Boolean Operators

Operator	Meaning	Example	Result
`and`	Both true	`True and False`	`False`
`or`	At least one true	`True or False`	`True`
`not`	Invert truth	`not True`	`False`

Key Takeaway

- ✅ Boolean values: `True` and `False` (capitalized)
- ✅ Comparison operators return boolean values
- ✅ `and`, `or`, `not` are boolean operators
- ✅ Short-circuit evaluation optimizes performance

Topic 1.7: Comparison and Boolean Operators 🔥

Overview

Understanding operator precedence and mixing boolean and comparison operators.

Operator Precedence

From highest to lowest:

Priority	Operators
1	`**`
2	`*`, `/`, `%`, `//`
3	`+`, `-`
4	`>`, `<`, `>=`, `<=`
5	`==`, `!=`
6	`not`
7	`and`
8	`or`

Mixing Boolean and Comparison Operators

```
x = 10
print(x > 5 and x < 20)    # True
print(x < 5 or x > 20)    # False
print(not x == 10)        # False
```

Key Takeaway

- ✅ `not` has highest boolean operator precedence
- ✅ `or` has lowest boolean operator precedence
- ✅ Use parentheses to clarify intent

Topic 1.8: Elements of Flow Control 🔥

Overview

Flow control determines the order in which statements are executed.

Conditional Statements

```python
x = 10

if x > 0:
    print("Positive")
elif x < 0:
    print("Negative")
else:
    print("Zero")
```

Loop Statements

for Loop

```python
# Iterate over a range
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4
```

while Loop

```python
count = 0
while count < 5:
    print(count)
    count += 1
```

Jump Statements

Statement	Description
`break`	Exit the loop entirely
`continue`	Skip to next iteration
`pass`	Do nothing (placeholder)

Key Takeaway

- ✅ `if`, `elif`, `else` for conditional execution
- ✅ `for` for definite iterations
- ✅ `while` for indefinite iterations
- ✅ `break`, `continue`, `pass` control loop flow

Topic 1.9: Program Execution 📚

Overview

Understanding how Python executes programs.

Program Execution Steps

1. **Load:** Python loads the program into memory
2. **Compile:** Source code converted to bytecode
3. **Interpret:** Bytecode executed line by line
4. **Output:** Results displayed

The name Variable

```
if __name__ == "__main__":
    print("This runs as main program")
```

Key Takeaway

- ✅ Python compiles to bytecode before execution
- ✅ `__name__ == "__main__"` checks if running as main

Topic 1.10: Flow Control Statements 🔥

Overview

Detailed flow control statements in Python.

if-elif-else Statement

```
if condition1:
    # Block 1
    pass
elif condition2:
    # Block 2
    pass
else:
    # Default block
    pass
```

for Loop with else

```
for i in range(5):
    print(i)
else:
    print("Loop completed")
```

while Loop with else

```
while count < 3:
    print(count)
    count += 1
else:
    print("While loop finished")
```

Key Takeaway

- ✅ Use `if-elif-else` for multiple conditions
- ✅ `else` block executes after loop completes normally

Topic 1.11: Importing Modules ⭐

Overview

Modules are files containing Python code that can be imported.

Ways to Import Modules

```python
# Import entire module
import math
print(math.sqrt(16))  # 4.0

# Import with alias
import math as m

# Import specific items
from math import sqrt, pi
```

Standard Library Modules

Module	Description
`math`	Mathematical functions
`random`	Random number generation
`datetime`	Date and time
`os`	Operating system interface

Key Takeaway

- ✅ Use `import` to include modules
- ✅ Use `from...import` for specific items

Topic 1.12: Ending a Program Early with sys.exit() 📚

Overview

Using sys.exit() to terminate program execution.

sys.exit()

```
import sys

print("Starting program...")
sys.exit()  # Exit immediately
```

Exit Codes

Code	Meaning
0	Successful termination
1	General error

Key Takeaway

- ✅ `sys.exit()` terminates the program
- ✅ Exit code 0 = success, non-zero = error

Unit 2: Functions, Lists, Dictionaries & Strings

Topic 2.1: Functions - Definition and Parameters 🔥

Overview

Functions are reusable blocks of code that perform specific tasks.

Defining Functions

```python
def function_name():
    """Docstring - describes the function"""
    pass
```

Function with Parameters

```python
# Single parameter
def greet(name):
    print(f"Hello, {name}!")

# Multiple parameters
def add(a, b):
    return a + b
```

Default Parameters

```python
def greet(name="Guest"):
    print(f"Hello, {name}!")
```

Variable Number of Arguments

```python
# *args - variable positional arguments
def sum_all(*args):
    total = 0
    for num in args:
        total += num
    return total

# **kwargs - variable keyword arguments
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
```

Key Takeaway

- ✅ Use `def` to define functions
- ✅ Parameters can have default values

- ✅ `*args` for variable positional arguments
- ✅ `**kwargs` for variable keyword arguments

Topic 2.2: Return Values and Return Statements 🔥

Overview

Return statements send values back from functions.

Return Statement

```
def add(a, b):
    return a + b

result = add(5, 3)  # result = 8
```

Returning Multiple Values

```
def get_stats(numbers):
    return min(numbers), max(numbers), sum(numbers)/len(numbers)

min_val, max_val, avg = get_stats([1, 2, 3, 4, 5])
```

Key Takeaway

- ✅ `return` exits the function and sends back a value
- ✅ Functions can return multiple values as a tuple

Topic 2.3: The None Value ⭐

Overview

`None` represents the absence of a value.

None Type

```
result = None
print(type(result))  # <class 'NoneType'>
```

Checking for None

```
if value is None:
    print("Value is None")
```

Key Takeaway

- ✅ `None` is Python's null value
- ✅ Use `is None` instead of `== None`
- ✅ Functions without return return `None`

Topic 2.4: Keyword Arguments and print() 🔥

Overview

Keyword arguments and the print() function for I/O.

print() Function

```
# Basic usage
print("Hello, World!")

# Multiple arguments
print("Hello", "World", sep="-")  # Hello-World

# f-strings
name = "John"
print(f"My name is {name}")
```

Keyword Arguments

```
def describe_pet(animal, name):
    print(f"I have a {animal} named {name}")

describe_pet(animal="dog", name="Buddy")
```

Key Takeaway

- ✅ `print()` outputs to console
- ✅ Use `sep`, `end`, `f-strings` for formatting

Topic 2.5: Local and Global Scope 🔥

Overview

Understanding variable scope in Python.

Local Scope

Variables defined inside a function are local:

```
def my_function():
    local_var = 10
    print(local_var)
```

Global Scope

Variables defined outside functions are global:

```
global_var = 20

def my_function():
    print(global_var)  # Can access global
```

Key Takeaway

- ✅ Variables inside functions are local
- ✅ Variables outside functions are global
- ✅ Use different names to avoid shadowing

Topic 2.6: The global Statement ⭐

Overview

Using `global` to modify global variables inside functions.

global Keyword

```
x = 10

def modify_global():
    global x
    x = 20

print(x)  # 10
modify_global()
print(x)  # 20
```

Key Takeaway

- ✅ `global` allows modifying global variables
- ✅ Use sparingly - makes code harder to debug

Topic 2.7: Exception Handling 🔥

Overview

Handling errors gracefully using try-except blocks.

Exception Types

Exception	Description
`ZeroDivisionError`	Division by zero
`ValueError`	Invalid value
`TypeError`	Wrong type
`FileNotFoundError`	File doesn't exist
`IndexError`	List index out of range

try-except Block

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Handling Multiple Exceptions

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("Invalid input!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

try-else-finally

```
try:
    # Risky code
    pass
except Exception as e:
    print(f"An error occurred: {e}")
finally:
    print("This always executes")
```

Raising Exceptions

```python
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
```

Key Takeaway

- ✅ `try` block contains risky code
- ✅ `except` catches specific exceptions
- ✅ `finally` always executes
- ✅ Use `raise` to throw exceptions

Topic 2.8: Lists - The List Data Type 🔥

Overview

Lists are ordered, mutable collections.

Creating Lists

```python
# Empty list
empty = []

# With initial values
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
```

Accessing Elements

```
fruits = ["apple", "banana", "cherry"]

# Positive index
print(fruits[0])   # apple

# Negative index
print(fruits[-1])  # cherry (last)
```

Key Takeaway

- ✅ Lists use square brackets `[]`
- ✅ Zero-indexed (first element is index 0)
- ✅ Negative indexing starts from -1

Topic 2.9: Working with Lists 🔥

Overview

Manipulating lists with slicing, adding, and removing elements.

Slicing

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:7])   # [2, 3, 4, 5, 6]
print(numbers[:5])    # [0, 1, 2, 3, 4]
print(numbers[5:])    # [5, 6, 7, 8, 9]
print(numbers[::2])   # [0, 2, 4, 6, 8]
print(numbers[::-1])  # Reversed
```

Adding Elements

```
fruits = ["apple", "banana"]
fruits.append("cherry")
fruits.insert(1, "mango")
fruits.extend(["grape", "orange"])
```

Removing Elements

```
fruits = ["apple", "banana", "cherry"]
fruits.remove("banana")
fruits.pop()
del fruits[0]
fruits.clear()
```

Key Takeaway

- ✅ Use slicing `[start:end]` to get portions
- ✅ `append()`, `insert()`, `extend()` to add
- ✅ `remove()`, `pop()`, `del` to remove

Topic 2.10: Augmented Assignment Operators ⭐

Overview

Shorthand operators that combine operation with assignment.

Operators

Operator	Equivalent	Example
`+=`	`x = x + y`	`x += 3`
`-=`	`x = x - y`	`x -= 3`
`*=`	`x = x * y`	`x *= 2`
`/=`	`x = x / y`	`x /= 4`

Examples

```
x = 10
x += 5   # x = 15
x -= 3   # x = 12
x *= 2   # x = 24

# With strings
s = "Hello"
s += " World"  # "Hello World"
```

Key Takeaway

- ✅ Shorthand for operation + assignment
- ✅ Works with numbers, strings, lists

Topic 2.11: List Methods 🔥

Overview

Built-in methods for list manipulation.

Common Methods

Method	Description
`append(item)`	Add to end
`insert(i, item)`	Insert at index
`extend(iterable)`	Add multiple
`remove(item)`	Remove first occurrence
`pop(i)`	Remove at index
`sort()`	Sort in place
`reverse()`	Reverse in place
`copy()`	Create shallow copy

List Comprehensions

```
squares = [x**2 for x in range(10)]
print(squares)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

evens = [x for x in range(10) if x % 2 == 0]
print(evens)  # [0, 2, 4, 6, 8]
```

Key Takeaway

- ✅ Most list methods modify in place
- ✅ List comprehensions create lists concisely

Topic 2.12: Dictionaries - The Dictionary Data Type 🔥

Overview

Dictionaries store data in key-value pairs.

Creating Dictionaries

```
# Empty dictionary
empty = {}

# With key-value pairs
person = {
    "name": "John",
    "age": 25,
    "city": "NYC"
}
```

Accessing Values

```
person = {"name": "John", "age": 25}

# Using key
print(person["name"])  # John

# Using get() - safer
print(person.get("name"))    # John
print(person.get("country")) # None (no error)
print(person.get("country", "USA"))  # USA (default)
```

Modifying Dictionaries

```
person = {"name": "John"}

# Add/update
person["age"] = 25
person.update({"city": "NYC", "age": 26})

# Remove
del person["city"]
age = person.pop("age")
```

Dictionary Methods

Method	Description
`keys()`	All keys
`values()`	All values
`items()`	All key-value pairs
`get(key)`	Get value safely

Key Takeaway

- ✅ Dictionaries use `{key: value}` syntax
- ✅ Keys must be unique and immutable
- ✅ Fast lookup by key (O(1) average)

Topic 2.13: Pretty Printing 📚

Overview

Using pprint module for readable output.

pprint Module

```
import pprint

data = {
    "name": "John Doe",
    "age": 30,
    "address": {
        "street": "123 Main St",
        "city": "Boston"
    }
}

pprint.pprint(data)
```

Key Takeaway

- ✅ `pprint` for readable output
- ✅ `pformat()` returns string

Topic 2.14: Using Data Structures to Model Real-World Things ⭐

Overview

Using data structures to represent real-world entities.

Example: Student Record

```
student = {
    "name": "John Doe",
    "id": "S12345",
    "gpa": 3.8,
    "courses": ["Python", "Math", "Physics"],
    "address": {
        "street": "123 Main St",
        "city": "Boston",
        "zip": "02101"
    }
}
```

Key Takeaway

- ✅ Use appropriate data structures for modeling
- ✅ Nested structures for complex data

Topic 2.15: Manipulating Strings 🔥

Overview

String indexing, slicing, and basic operations.

String Indexing and Slicing

```
text = "Hello, World!"

# Indexing
print(text[0])    # H
print(text[-1])   # !

# Slicing
print(text[0:5])   # Hello
print(text[7:])    # World!
print(text[::-1])  # Reversed
```

String Concatenation

```
first = "Hello"
second = "World"
result = first + " " + second
```

Key Takeaway

- ✅ Strings are immutable (can't change in place)
- ✅ Use slicing for substrings

Topic 2.16: Useful String Methods 🔥

Overview

Common string methods for text manipulation.

Case Conversion

```
text = "Hello World"

print(text.upper())      # HELLO WORLD
print(text.lower())      # hello world
print(text.title())      # Hello World
```

Searching and Finding

```
text = "Hello, World!"

print(text.find("World"))     # 7 (index)
print(text.count("l"))       # 3
print(text.startswith("Hello"))  # True
```

Splitting and Joining

```
text = "apple,banana,cherry"

# Split
parts = text.split(",")
print(parts)  # ['apple', 'banana', 'cherry']

# Join
result = " - ".join(parts)
```

Key Takeaway

- ✅ Use `split()` and `join()` for word manipulation
- ✅ Check character types with `is*` methods

Other Important Concepts (Frequently Asked in Exams) 🔥

Topic 3.1: Tuples 🔥

Overview

Tuples are ordered, immutable sequences.

Creating Tuples

```python
# Empty tuple
empty = ()

# With values
coordinates = (10, 20)
mixed = (1, "hello", 3.14)

# Single element (needs comma)
single = (5,)  # NOT (5)
```

Tuple Features

Feature	Description
Ordered	Elements have defined order
Immutable	Cannot be modified after creation
Indexed	Zero-based indexing
Heterogeneous	Can hold different types

Accessing Elements

```python
coordinates = (10, 20, 30)

print(coordinates[0])   # 10
print(coordinates[-1])  # 30
print(coordinates[1:3]) # (20, 30)
```

Tuple Methods

```python
my_tuple = (1, 2, 3, 2, 4)

print(my_tuple.count(2))  # 2
print(my_tuple.index(3))   # 2
```

Tuple Packing and Unpacking

```python
# Packing
point = 10, 20, 30

# Unpacking
x, y, z = point
print(x, y, z)  # 10 20 30
```

Key Differences: List vs Tuple

Feature	List	Tuple
Mutable	Yes	No
Syntax	`[ ]`	`( )`
Performance	Slower	Faster
Memory	More	Less

Key Takeaway

- ✅ Tuples use parentheses `( )`
- ✅ Immutable - cannot change after creation
- ✅ Used for fixed collections and function return values

Topic 3.2: Sets ⭐

Overview

Sets are unordered collections of unique elements.

Creating Sets

```
# Empty set (NOT {})
empty_set = set()

# With values
fruits = {"apple", "banana", "cherry"}

# From list (removes duplicates)
numbers = set([1, 2, 3, 2, 1])
print(numbers)  # {1, 2, 3}
```

Set Operations

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

# Union
print(set1 | set2)        # {1, 2, 3, 4, 5, 6}
print(set1.union(set2))  # {1, 2, 3, 4, 5, 6}

# Intersection
print(set1 & set2)        # {3, 4}
print(set1.intersection(set2))  # {3, 4}

# Difference
print(set1 - set2)        # {1, 2}
print(set1.difference(set2))  # {1, 2}

# Symmetric Difference
print(set1 ^ set2)        # {1, 2, 5, 6}
```

Set Methods

```
fruits = {"apple", "banana"}

fruits.add("cherry")       # Add single
fruits.update(["mango", "grape"])  # Add multiple
fruits.remove("banana")    # Remove (raises error if not found)
fruits.discard("orange")   # Remove (no error if not found)
fruits.pop()               # Remove random element
fruits.clear()             # Clear all
```

Set Comprehension

```
squares = {x**2 for x in range(5)}
print(squares)  # {0, 1, 4, 9, 16}
```

Key Takeaway

- ✅ Sets use curly braces `{ }`
- ✅ No duplicate elements
- ✅ Unordered - no indexing
- ✅ Great for membership testing and math operations

Topic 3.3: List Comprehensions 🔥

Overview

List comprehensions provide a concise way to create lists.

Basic Syntax

```
# Traditional way
squares = []
for x in range(5):
    squares.append(x**2)

# List comprehension
squares = [x**2 for x in range(5)]
# [0, 1, 4, 9, 16]
```

With Condition

```
# Get even numbers
evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]

# Get squares of even numbers
even_squares = [x**2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]
```

Nested List Comprehension

```
# Flatten a matrix
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [num for row in matrix for num in row]
# [1, 2, 3, 4, 5, 6]
```

With if-else

```
# Replace values
result = [x if x > 0 else 0 for x in [-1, 2, -3, 4]]
# [0, 2, 0, 4]
```

Examples

```
# Character extraction
text = "Hello"
chars = [c for c in text]
# ['H', 'e', 'l', 'l', 'o']

# Matrix transpose
matrix = [[1, 2], [3, 4], [5, 6]]
transpose = [[row[i] for row in matrix] for i in range(2)]
# [[1, 3, 5], [2, 4, 6]]
```

Key Takeaway

- ✅ Concise syntax: `[expression for item in iterable if condition]`

- ✅ More readable than traditional loops
- ✅ Faster execution than append in loops

Topic 3.4: Dictionary Comprehensions 🔥

Overview

Dictionary comprehensions create dictionaries concisely.

Basic Syntax

```
# Traditional way
squares = {}
for x in range(5):
    squares[x] = x**2

# Dictionary comprehension
squares = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

With Condition

```
# Only even keys
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
# {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

From Two Lists

```
keys = ['a', 'b', 'c']
values = [1, 2, 3]

d = {k: v for k, v in zip(keys, values)}
# {'a': 1, 'b': 2, 'c': 3}
```

Swap Keys and Values

```
original = {'a': 1, 'b': 2, 'c': 3}
swapped = {v: k for k, v in original.items()}
# {1: 'a', 2: 'b', 3: 'c'}
```

Examples

```
# Word length dictionary
words = ['apple', 'banana', 'cherry']
word_lengths = {word: len(word) for word in words}
# {'apple': 5, 'banana': 6, 'cherry': 6}

# Filter dictionary
scores = {'Alice': 85, 'Bob': 92, 'Charlie': 78, 'Diana': 95}
passing = {name: score for name, score in scores.items() if score >= 80}
# {'Alice': 85, 'Bob': 92, 'Diana': 95}
```

Key Takeaway

- ✅ Syntax: `{key: value for item in iterable if condition}`
- ✅ Efficient way to create dictionaries
- ✅ Can transform existing dictionaries

Topic 3.5: Lambda Functions 🔥

Overview

Lambda functions are small anonymous functions.

Lambda Syntax

```python
# Traditional function
def square(x):
    return x ** 2

# Lambda function
square = lambda x: x ** 2

print(square(5))  # 25
```

Lambda with Multiple Arguments

```python
# Addition
add = lambda a, b: a + b
print(add(3, 5))  # 8

# Maximum
max_val = lambda a, b: a if a > b else b
print(max_val(10, 5))  # 10
```

Common Use Cases

Sorting

```python
# Sort by length
fruits = ['apple', 'banana', 'cherry', 'date']
sorted_fruits = sorted(fruits, key=lambda x: len(x))
# ['date', 'apple', 'cherry', 'banana']

# Sort list of tuples
points = [(1, 2), (3, 1), (2, 3)]
sorted_points = sorted(points, key=lambda x: x[1])
# [(3, 1), (1, 2), (2, 3)]
```

with map()

```python
numbers = [1, 2, 3, 4, 5]

# Square all numbers
squares = list(map(lambda x: x**2, numbers))
# [1, 4, 9, 16, 25]
```

with filter()

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Get even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [2, 4, 6, 8, 10]
```

Key Takeaway

- ✅ Syntax: `lambda arguments: expression`
- ✅ Anonymous functions (no name)
- ✅ Used with `map()`, `filter()`, `sorted()`
- ✅ Keep lambdas simple - use def for complex logic

Topic 3.6: Map, Filter, and Reduce 🔥

Overview

Functional programming tools for processing iterables.

map() Function

Applies a function to all items in an iterable.

```
# Syntax: map(function, iterable)

numbers = [1, 2, 3, 4, 5]

# Square all numbers
squares = list(map(lambda x: x**2, numbers))
print(squares)  # [1, 4, 9, 16, 25]

# With regular function
def double(x * 2

doubled = list(map(double, numbers):
    return x))
print(doubled)  # [2, 4, 6, 8, 10]
```

filter() Function

Filters items based on a condition.

```
# Syntax: filter(function, iterable)

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Get even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # [2, 4, 6, 8, 10]

# Get numbers greater than 5
greater = list(filter(lambda x: x > 5, numbers))
print(greater)  # [6, 7, 8, 9, 10]
```

reduce() Function

Reduces an iterable to a single value.

```
# Syntax: reduce(function, iterable)

from functools import reduce

numbers = [1, 2, 3, 4, 5]

# Sum all numbers
total = reduce(lambda a, b: a + b, numbers)
print(total)  # 15

# Product of all numbers
product = reduce(lambda a, b: a * b, numbers)
print(product)  # 120

# Maximum
maximum = reduce(lambda a, b: a if a > b else b, numbers)
print(maximum)  # 5
```

Combining map, filter, and reduce

```python
from functools import reduce

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Chain: Square even numbers, then sum
result = reduce(
    lambda a, b: a + b,
    map(lambda x: x**2, filter(lambda x: x % 2 == 0, numbers))
)
print(result)  # 220 (4 + 16 + 36 + 64 + 100)
```

Key Takeaway

- ✅ `map()` : Transform each element
- ✅ `filter()` : Select elements based on condition
- ✅ `reduce()` : Combine elements into single value
- ✅ Enable functional programming in Python

Topic 3.7: File Handling 🔥

Overview

Reading from and writing to files.

Opening Files

```python
# Different modes
# 'r' - read (default)
# 'w' - write (overwrites)
# 'a' - append
# 'rb' - read binary
# 'wb' - write binary

file = open("example.txt", "r")
```

Reading Files

```python
# Read entire file
with open("example.txt", "r") as file:
    content = file.read()
    print(content)

# Read line by line
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())

# Read all lines into list
with open("example.txt", "r") as file:
    lines = file.readlines()

# Read single line
with open("example.txt", "r") as file:
    first_line = file.readline()
```

Writing Files

```python
# Write to file
with open("example.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("Second line")

# Append to file
with open("example.txt", "a") as file:
    file.write("\nAppended line")

# Write multiple lines
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("example.txt", "w") as file:
    file.writelines(lines)
```

File Methods

Method	Description
`read()`	Read entire file
`readline()`	Read single line
`readlines()`	Read all lines into list
`write()`	Write string
`writelines()`	Write list of strings
`close()`	Close file

Working with CSV Files

```python
import csv

# Read CSV
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# Write CSV
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age"])
    writer.writerow(["John", "25"])
    writer.writerow(["Alice", "30"])
```

Key Takeaway

- ✅ Always use `with` statement for file handling
- ✅ Close files automatically
- ✅ Handle exceptions with try-finally

Topic 3.8: Regular Expressions 🔥

Overview

Regular expressions (regex) are patterns used for matching text.

Import re Module

```
import re
```

Basic Patterns

Pattern	Description	Example
`.`	Any character	`a.c` matches "abc"
`\d`	Digit (0-9)	`\d+` matches "123"
`\D`	Non-digit	`\D+` matches "abc"
`\w`	Word character	`\w+` matches "hello"
`\W`	Non-word	`\W+` matches "@#"
`\s`	Whitespace	`\s+` matches " "
`\S`	Non-whitespace	`\S+` matches "hello"

Quantifiers

Quantifier	Description
`*`	Zero or more
`+`	One or more
`?`	Zero or one
`{n}`	Exactly n times
`{n,}`	n or more times
`{n,m}`	Between n and m times

re Functions

```python
import re

text = "My phone number is 123-456-7890"

# search() - Find first match
match = re.search(r'\d{3}-\d{3}-\d{4}', text)
print(match.group())  # 123-456-7890

# findall() - Find all matches
numbers = re.findall(r'\d+', text)
print(numbers)  # ['123', '456', '7890']

# sub() - Replace
new_text = re.sub(r'\d{3}-\d{3}-\d{4}', 'XXX-XXX-XXXX', text)
print(new_text)  # My phone number is XXX-XXX-XXXX

# split() - Split by pattern
words = re.split(r'\s+', text)
print(words)  # ['My', 'phone', 'number', 'is', '123-456-7890']
```

Metacharacters

Metacharacter	Description
`^`	Start of string
`$`	End of string
`[]`	Character class
`	`
`()`	Grouping

Examples

```python
import re

# Validate email
email = "user@example.com"
pattern = r'^[\w.-]+@[\w.-]+\.\w+$'
print(re.match(pattern, email))  # Match object

# Validate phone
phone = "123-456-7890"
pattern = r'^\d{3}-\d{3}-\d{4}$'
print(re.match(pattern, phone))  # Match object

# Extract dates
text = "Today is 2024-01-15, tomorrow is 2024-01-16"
dates = re.findall(r'\d{4}-\d{2}-\d{2}', text)
print(dates)  # ['2024-01-15', '2024-01-16']
```

Key Takeaway

- ✅ Use `re.search()`, `re.findall()`, `re.sub()`
- ✅ Patterns use special characters: `.`, `*`, `+`, `^`, `$`
- ✅ `\d` for digits, `\w` for words, `\s` for spaces

Topic 3.9: Object-Oriented Programming - Classes and Objects 🔥

Overview

Classes are blueprints for creating objects.

Creating a Class

```python
class Person:
    # Class attribute
    species = "Human"

    # Constructor (__init__)
    def __init__(self, name, age):
        # Instance attributes
        self.name = name
        self.age = age

    # Instance method
    def greet(self):
        return f"Hello, I'm {self.name}"

    # String representation
    def __str__(self):
        return f"Person: {self.name}, {self.age}"
```

Creating Objects

```python
# Create object
person1 = Person("John", 25)
person2 = Person("Alice", 30)

# Access attributes
print(person1.name)  # John
print(person1.species)  # Human

# Call methods
print(person1.greet())  # Hello, I'm John
print(person1)  # Person: John, 25
```

Class vs Instance Attributes

```python
class Dog:
    # Class attribute (shared by all instances)
    species = "Canis familiaris"

    def __init__(self, name, age):
        # Instance attributes (unique to each instance)
        self.name = name
        self.age = age
```

Methods

```python
class Calculator:
    # Instance method
    def add(self, a, b):
        return a + b

    # Class method (use @classmethod)
    @classmethod
    def info(cls):
        return "Calculator class"

    # Static method (no self or cls)
    @staticmethod
    def multiply(a, b):
        return a * b
```

Key Takeaway

- ✅ Use `class` keyword to define classes
- ✅ `__init__` is the constructor
- ✅ `self` refers to the instance

Topic 3.10: Inheritance 🔥

Overview

Inheritance allows a class to inherit attributes and methods from another class.

Basic Inheritance

```python
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some sound"

# Child class
class Dog(Animal):
    def speak(self):
        return "Woof!"

# Create object
dog = Dog("Buddy")
print(dog.name)  # Buddy
print(dog.speak())  # Woof!
```

Multiple Inheritance

```python
class A:
    def method_a(self):
        return "A"

class B:
    def method_b(self):
        return "B"

class C(A, B):
    def method_c(self):
        return "C"

obj = C()
print(obj.method_a())  # A
print(obj.method_b())  # B
print(obj.method_c())  # C
```

Method Overriding

```python
class Parent:
    def greet(self):
        return "Hello from Parent"

class Child(Parent):
    def greet(self):
        return "Hello from Child"

obj = Child()
print(obj.greet())  # Hello from Child
```

super() Function

```python
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "Canine")  # Call parent constructor
        self.breed = breed
```

Key Takeaway

- ✅ Use `class Child(Parent)` for inheritance
- ✅ Child class inherits all parent methods
- ✅ Use `super()` to call parent methods

Topic 3.11: Generators ⭐

Overview

Generators are functions that yield values lazily (one at a time).

Generator Function

```python
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

# Create generator
gen = count_up_to(5)

# Iterate
for num in gen:
    print(num)  # 1, 2, 3, 4, 5
```

Generator vs List

```python
# List (stores all values)
def squares_list(n):
    return [x**2 for x in range(n)]

# Generator (yields one at a time)
def squares_gen(n):
    for x in range(n):
        yield x**2
```

Generator Expression

```python
# Like list comprehension but with ()
squares = (x**2 for x in range(5))

for square in squares:
    print(square)
```

next() Function

```python
def simple_gen():
    yield 1
    yield 2
    yield 3

gen = simple_gen()
print(next(gen))  # 1
print(next(gen))  # 2
print(next(gen))  # 3
# print(next(gen))  # StopIteration
```

Key Takeaway

- ✅ Use `yield` instead of `return`
- ✅ Memory efficient (don't store all values)
- ✅ Can be iterated only once

Topic 3.12: Decorators ⭐

Overview

Decorators modify the behavior of functions or classes.

Basic Decorator

```python
def my_decorator(func):
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
# Output:
# Before function
# Hello!
# After function
```

Decorator with Arguments

```python
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def greet(name):
    print(f"Hello, {name}!")

greet("John")
# Hello, John!
# Hello, John!
# Hello, John!
```

Built-in Decorators

```python
class MyClass:
    name = "Class"

    @staticmethod
    def static_method():
        return "Static"

    @classmethod
    def class_method(cls):
        return f"Class: {cls.name}"

    @property
    def prop(self):
        return "Property"
```

Key Takeaway

- ✅ Use `@decorator` syntax
- ✅ Modify function behavior without changing code
- ✅ Common use: logging, timing, authentication

Topic 3.13: Modules and Packages ⭐

Overview

Modules are Python files; packages are directories containing modules.

Creating a Module

```python
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

Using a Module

```python
import mymodule

print(mymodule.greet("John"))  # Hello, John!
print(mymodule.add(3, 5))  # 8

# Import specific
from mymodule import greet, add

# Import with alias
import mymodule as mm
```

Creating a Package

```
mypackage/
    __init__.py
    module1.py
    module2.py
    subpackage/
        __init__.py
        module3.py
```

init.py

```python
# __init__.py
from .module1 import function1
from .module2 import function2

__all__ = ['function1', 'function2']
```

Using Package

```python
from mypackage import function1
from mypackage.subpackage import module3
```

Standard Library Modules

```
import datetime  # Date and time
import json      # JSON handling
import os        # Operating system
import math       # Mathematical functions
import random     # Random numbers
import re         # Regular expressions
```

Key Takeaway

- ✅ Modules are .py files
- ✅ Packages are directories with `__init__.py`
- ✅ Use `import` to include modules

📊 Coverage Statistics

Metric	Value
Total Topics (Unit 1)	12
Total Topics (Unit 2)	16
Additional Important Topics	13
Total Topics	**41**
Coverage	**100%**

🎯 Exam Priority Guide

🔥 High Priority Topics (Frequently Asked)

- Introduction to Python
- Data Types and Variables

- Flow Control (if-else, loops)
- Functions and Parameters
- Lists and List Methods
- Dictionaries
- Strings and String Methods
- Exception Handling
- List/Dictionary Comprehensions
- Lambda Functions
- Map, Filter, Reduce
- File Handling
- Regular Expressions
- OOP (Classes and Inheritance)

⭐ Medium Priority Topics

- String Operations
- Tuples
- Sets
- Modules and Packages
- Generators
- Decorators

📚 Standard Topics

- Pretty Printing
- Program Execution
- Dissecting Programs

Notes compiled for Python Unit 1 and Unit 2 - Complete syllabus coverage with additional exam-focused topics.